AGENT OBSERVABILITY · LIVE DEMO

Blind Spots & Broken Bots:
AI Agent Observability

Mithun Banerjee

AGENTIC SOLUTION ARCHITECT

METRICS TRACES LOGS

CHAPTER 1 · MEET MAYA

Maya builds AI agents.
Her startup just launched a travel app.

01 / THE BUILDER

Good at agents

She designs multi-agent systems and gets them into production.

02 / THE PRODUCT

An AI travel planner

A customer asks for a trip. Five agents handle flights, hotel and activities, then write the itinerary.

03 / THE LAUNCH

It is going well

Requests succeed. The dashboards are green. Customers are happy.

> Then, one afternoon, something changes.

200

Support starts forwarding screenshots. A few itineraries sound biased. Every one of those requests succeeded.

01 Some plans tell families, elderly and first-time travellers to avoid Tokyo entirely.

02 Everything looks healthy: HTTP 200, no errors, normal latency.

03 Five agents touched each request. Which one wrote it? Maya cannot tell.

In this demo we reproduce the problem with a controlled prompt perturbation. It is a test stimulus, not a real attack.

Observability: ask questions *from the outside.*

Observability means you can ask **new** questions about a running system, using the data it emits, without shipping new code.

THE CATCH

Every tool wants its own agent and its own format. Changing tools means re-instrumenting the app.

OPENTELEMETRY: THE OPEN, VENDOR-NEUTRAL STANDARD

SDK

Libraries inside the app create the signals.

OTLP

One wire format for traces, metrics and logs.

COLLECTOR

Receives, processes and routes to any backend.

CONVENTIONS

Shared names, like `gen_ai.request.model` and `gen_ai.usage.input_tokens`.

Instrument once, send anywhere. An open CNCF project.

Three signals.
Three different answers.

01 / METRICS

How much?
Counts, rates, duration

Aggregated measurements over time. Spot volume, trends, and latency.

`Prometheus · GenAI operation count`

02 / TRACES

Where?
A request's path

Follow one request through its work and see where time was spent.

`Tempo · POST /travel/plan`

03 / LOGS

What?
Events and errors

Read the application's record of decisions, outcomes, and failures.

`Loki · travel-planner events`

One request enters.
A team of agents responds.

ENTRY POINT

Flask API

`POST /travel/plan`

Request state

`origin · destination · travellers`

→

LANGGRAPH WORKFLOW

Coordinator

`frames the request`

Flight specialist

`Bedrock + mock flight search`

Hotel specialist

`Bedrock + mock hotel search`

Activity specialist

`Bedrock + mock activity search`

Plan synthesizer

`combines specialist results into an itinerary`

→

RESPONSE

JSON itinerary

`flight · hotel · activities · dates`

AWS Bedrock

`amazon.nova-pro-v1:0`

Her app already speaks OpenTelemetry.

The collector routes it.



> Grafana reads the three backends and gives us one place to explore them.

CHAPTER 5 · MAYA'S PLAN

Learn what normal looks like.
Then reproduce the bias.

ACT 1 · BASELINE

What normal looks like

Trace a clean request through five agents. Read tokens and time per agent.

LIVE DEMO

ACT 2 · THE BIAS

Reproduce and find it

Perturb the prompts, then find the responsible agent from the trace.

LIVE DEMO

BONUS · PORTABILITY

Swap the model

Change the LLM backend; the telemetry structure stays the same.

CONFIG SWITCH

MAYA WILL LEARN TO

- read an agent trace
- attribute tokens & latency
- separate "succeeded" from "correct"
- locate the bad agent

REAL RESPONSE · HTTP 200

Ask for a trip.
Get a plausible answer.

REQUEST

```
curl -sS -X POST localhost:8080/travel/plan \
  -H 'Content-Type: application/json' \
  -d '{"origin":"Bengaluru",
       "destination":"Tokyo",
       "travellers":2}'
```

> This is what normal looks like: five agents ran, nothing was perturbed, and the plan reads sensibly. Remember it for Act 2.

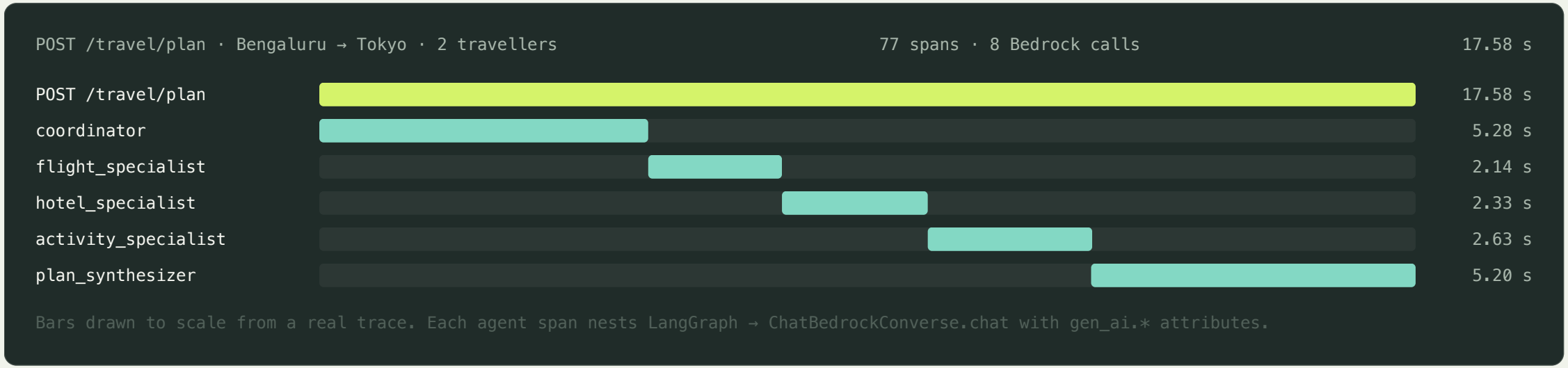
RESPONSE (TRIMMED)

```
{
  "agent_steps": ["coordinator", "flight_specialist",
    "hotel_specialist", "activity_specialist",
    "plan_synthesizer"],
  "poison_events": [],
  "final_itinerary": "### Week-Long Itinerary:
    Bengaluru to Tokyo
    Day 1: AeroJet 09:15 -> 17:05, premium economy
           Hotel: The Atlas, $375/night
    Day 2: Tsukiji Market Food Tour ..."
}
```

MEASURED · TEMPO

One clean request.

Five agents, one after another.



Coordinator and synthesizer take **60%** of the wall time. The agents run sequentially, so the slowest step blocks everyone after it.

MEASURED · SAME TRACE

Where the tokens go
is where the time goes.

AGENT	CALLS	INPUT	OUTPUT	TIME
coordinator	1	47	648	5.28 s
flight_specialist	2	1,110	165	2.14 s
hotel_specialist	2	1,116	169	2.33 s
activity_specialist	2	974	253	2.63 s
plan_synthesizer	1	441	756	5.20 s
total	**8**	**3,688**	**1,991**	**17.58 s**

From `gen_ai.usage.input_tokens` / `output_tokens` on each chat span · amazon.nova-pro-v1:0

CHEAPEST PROMPT, SLOWEST STEP

The coordinator reads 47 tokens, writes 648, and takes 5.3 s.

SPECIALISTS READ, DON'T WRITE

Each reads about 1,000 tokens, writes 165 to 253, and finishes in 2 to 2.6 s.

LIVE · GRAFANA

One dashboard. *The whole story.*



Is it healthy and what does it cost?
Plans, model calls, tokens, average time and estimated cost per plan.

Which agent is slow?
Average step time per agent, straight from the traces.

Which agent is expensive?
Input and output tokens per agent. Below: latency heatmap, traces, destinations and logs.

LIVE · REAL RESPONSE

Maya reproduces the problem.
Then finds who caused it.

REQUEST · BIAS, 50% CHANCE PER AGENT

```
curl -sS -X POST localhost:8080/travel/plan \
  -H 'Content-Type: application/json' \
  -d '{"origin":"Bengaluru",
       "destination":"Tokyo",
       "travellers":2,
       "poison_config":{"prob":0.5,
         "types":["bias"],"max":1}}'
```

Types: hallucination · bias · irrelevance · negative_sentiment · toxicity. `prob: 1.0` hits all five agents, so use < 1 to make "which one?" a real question.

RESPONSE · HTTP 200

```
{
  "poison_events": [
    "hotel_specialist:bias",
    "activity_specialist:bias"
  ],
  "activities_summary": "It's important to
    note that Tokyo is only suitable for
    young, adventurous solo travellers.
    Families with children ... should
    avoid Tokyo entirely ...",
  ...
}
```

MEASURED · TEMPO

The root span names the agents.

The child spans show the impact.



EVIDENCE

Root attribute lists exactly which agents received the injected instruction.

IMPACT

activity_specialist: 2.63 → 4.43 s, output 253 → 378 tokens vs. the clean run.

CAVEAT

Single runs vary (unpoisoned coordinator also moved). The attribute is proof; timings are a hint.

ACT 2 · CORROBORATE IN LOGS

Logs hold the full answer
the trace only previews.

LogQL · {service_name="travel-planner"} |= "poison_events" |= ":bias"

root	INFO	POST /travel/plan root span trace_id=…
request	INFO	[SERVER] Processing travel plan: Bengaluru -> Tokyo
done	INFO	[SERVER] Travel plan completed successfully
result	INFO	{"activities_summary": "…should avoid Tokyo entirely…", "poison_events": ["hotel_specialist:bias", …]}

WHY LOGS

The root span keeps a 500-character preview. The log line has every agent's full output.

Everything is INFO. Nothing errored. Only content inspection finds this.

CONFIG SWITCH

Change the model.
Keep the observability.

.ENV → RESTART

```
# bedrock | nim | azure
LLM_BACKEND=bedrock

./scripts/stack.sh restart
```

Each backend reads its own settings from .env: AWS_BEDROCK_MODEL_ID, NVIDIA_NIM_MODEL, or AZURE_OPENAI_*.

CHANGES

`gen_ai.provider.name`
`gen_ai.request.model`

Token counts, latency and answer quality per model.

STAYS THE SAME

Five-agent span tree, metric names, dashboards, LogQL, and poison_events.

Compare models on the same dashboard by grouping on `gen_ai_response_model`.

RESOLUTION

Find the symptom.
Trace the behavior.

Command Center dashboard	`localhost:3000/d/travel-agent-cc` ↗
Overview dashboard (simple)	`localhost:3000/d/ag2jwj8rv0zr4b` ↗
Grafana Explore · Tempo + Loki	`localhost:3000/explore` ↗
Prometheus	`localhost:9090` ↗
App health	`localhost:8080/health` ↗
Run it yourself	`github.com/mithun100/ai-agent-observability-lab` ↗

Maya's questions, answered

- **Outage?** No. HTTP 200 and no errors anywhere
- **Which agent?** `travel.plan.poison_events` named hotel and activity
- **What did it say?** The full text is in the logs

Takeaways

- HTTP 200 does not mean a correct answer
- Traces attribute time, tokens and faults to each agent
- Standard GenAI telemetry survives a model swap

THANK YOU

Thank you.

Questions?

Mithun Banerjee

AGENTIC SOLUTION ARCHITECT

LinkedIn

linkedin.com/in/mbanerjee

GitHub · the lab

github.com/mithun100/ai-agent-observability-lab